1.INTRODUCTION

1.1OVERVIEW OF THE SYSTEM

The **Stray Dogs or Cats Management System** is a web-based application developed to improve the welfare and management of stray animals by connecting users, care centres, and veterinarians through a unified digital platform. The system is designed to simplify the process of reporting, rescuing, adopting, and providing medical assistance to stray dogs and cats.

This platform allows users to register and log in to access various services such as viewing available stray animals for adoption, adopting pets online, reporting stray animals in their locality, and requesting veterinary consultations. Users can also track their appointments and monitor the status of their requests, ensuring transparency and ease of communication.

The system includes a dedicated module for veterinarians, enabling them to manage consultation requests, schedule appointments, and provide medical assistance for both reported and adopted animals. Veterinarians can also update their profiles, making it easier for users and care centres to connect with qualified professionals.

Care centres play an essential role in the system by managing veterinarians, reviewing user requests, and adding stray animals to the adoption list. They are responsible for updating the status of animals and ensuring that accurate and up-to-date information is available on the platform.

An administrative module oversees the entire system, managing users, care centres, and veterinarians while ensuring smooth operation, data security, and system reliability.

Overall, the system provides an efficient and organized solution for stray animal management by integrating adoption services, reporting mechanisms, and veterinary support into a single platform, thereby promoting responsible pet adoption and improving animal welfare.

1.2PROBLEM DEFINITION AND OBJECTIVE OF THE PROJECT

Problem definition:

Stray dogs and cats are a common issue in many communities, leading to concerns related to animal welfare, public safety, and health. Currently, there is no proper centralized system to manage stray animal reporting, rescue, adoption, and medical assistance. Most processes are handled manually or through unorganized channels, which leads to delays, miscommunication, and lack of proper tracking.

People who wish to adopt stray animals often face difficulty in finding reliable information about available pets. Similarly, reporting stray animals or requesting veterinary help is not streamlined, making it hard for care centres and veterinarians to respond quickly and effectively. In addition, there is limited coordination between users, care centres, and veterinarians, resulting in inefficient handling of cases.

Due to these challenges, many stray animals do not receive timely care, and potential adopters are unable to connect with them easily. This highlights the need for a structured, transparent, and efficient system to address these issues.

Objectives of the Project

The main objectives of the project are:

- To develop a **centralized system** for managing stray dogs and cats.
- To provide an easy way for users to **report stray animals** in their locality.
- To create a platform for **adopting stray animals online**.
- To enable users to **request veterinary consultations** and schedule appointments.
- To assist care centres in **managing animal data and adoption processes**.
- To improve **communication and coordination** between users, veterinarians, and care centres.
- To ensure **transparency and tracking** of requests, reports, and appointments.
- To promote **animal welfare** by ensuring timely care and responsible adoption.

Overall, the project aims to solve existing problems in stray animal management by introducing an efficient, organized, and user-friendly digital platform.

2. SYSTEM ANALYSIS

2.1 INTRODUCTION

Stray animals, especially dogs and cats, are increasingly found in urban and rural areas, creating challenges related to animal welfare, public health, and safety. Many of these animals suffer from hunger, injuries, and lack of proper medical care. At the same time, people who are willing to help or adopt such animals often face difficulties due to the absence of a proper system to connect them with reliable sources.

With the advancement of technology, web-based applications have become an effective solution for managing real-world problems in an organized and efficient manner. The **Stray Dogs or Cats Management System** is developed as a digital platform to address the issues associated with stray animal management by providing a centralized system that connects users, veterinarians, care centres, and administrators.

This system simplifies various processes such as reporting stray animals, requesting veterinary assistance, scheduling appointments, and adopting pets. It ensures that all stakeholders can interact through a single platform, improving communication and coordination. The platform also provides transparency by allowing users to track the status of their requests and activities.

The introduction of such a system helps reduce manual efforts and improves the efficiency of handling stray animal cases. By integrating different services into one application, the system plays a significant role in promoting responsible pet adoption and ensuring timely medical care for stray animals.

Overall, this project demonstrates how technology can be used effectively to solve social issues and contribute to the welfare of animals in the community.

2.2 IDENTIFICATION OF NEED

The increasing number of stray dogs and cats in many areas has created a significant need for an efficient system to manage their rescue, care, and adoption. Currently, the absence of a centralized platform makes it difficult for people to report stray animals, seek medical assistance, or adopt pets in an organized manner.

In most cases, information about stray animals is scattered across different sources such as social media, local groups, or word of mouth. This lack of proper coordination leads to delays in responding to emergencies and reduces the chances of animals receiving timely care. Additionally, individuals who are willing to adopt pets often struggle to find accurate and updated information about available animals.

Care centres and veterinarians also face challenges in managing requests and maintaining proper communication with the public. Without a structured system, tracking reports, scheduling appointments, and updating the status of animals becomes difficult and inefficient.

There is also a need to ensure transparency and accountability in the process of reporting, treatment, and adoption. A well-organized system can help in monitoring activities, maintaining records, and improving overall efficiency.

2.3 EXISTING SYSTEM

The existing system for managing stray dogs and cats is mostly manual and unorganized. People usually rely on social media, local groups, or direct contact with care centres and veterinarians to report or adopt animals.

There is no centralized platform to manage all activities, which leads to poor communication, delays in response, and difficulty in tracking reports and requests. Care centres and veterinarians maintain records separately, making the process inefficient.

Overall, the current system lacks coordination, transparency, and efficiency in handling stray animal management.

2.3.1 LIMITATION OF EXISTING SYSTEM

The existing system for managing stray dogs and cats has several limitations:

- Lack of a centralized platform to manage all activities
- Poor communication and coordination between users, care centres, and veterinarians
- Difficulty in tracking reports, requests, and appointments
- Delays in providing timely medical assistance to animals
- Lack of updated and reliable information about stray animals
- Inefficient and time-consuming adoption process
- Dependence on manual work and informal methods like social media
- No proper record maintenance or data management
- Limited transparency in the overall process
- Difficulty in handling multiple requests simultaneously

These limitations reduce the efficiency of the existing system and highlight the need for a more organized and digital solution.

2.4 PROPOSED SYSTEM

The proposed system is a web-based application designed to provide a centralized platform for managing stray dogs and cats efficiently. It connects users, veterinarians, care centres, and administrators in a single system to improve coordination and communication.

The system allows users to report stray animals, adopt pets online, and request veterinary consultations. Care centres can manage animal records and update their status, while veterinarians can handle requests and schedule appointments. The admin monitors and controls all activities to ensure smooth functioning.

This system overcomes the limitations of the existing system by providing better **organization, transparency, and efficiency**, making stray animal management easier and more effective.

2.4.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages:

- Provides a centralized platform for all activities
- Improves communication and coordination between users, care centres, and veterinarians
- Enables easy reporting of stray animals
- Simplifies the pet adoption process
- Allows appointment scheduling and veterinary consultation
- Ensures better tracking of requests and status updates
- Reduces manual work and increases efficiency
- Maintains proper data records and management
- Provides transparency in all processes
- Enhances overall animal welfare and care

These advantages make the proposed system more efficient, reliable, and user-friendly compared to the existing system.

2.5 FEASIBILITY STUDY

The feasibility study shows that the proposed system is practical and easy to implement. It is developed using reliable and easily available technologies like Python and Django, making it technically feasible.

The system is cost-effective as it uses open-source tools and does not require expensive resources. It is also user-friendly, so users, care centres, and veterinarians can operate it easily.

In addition, the system can be developed within a reasonable time and provides social benefits by improving animal welfare and adoption processes.

Overall, the proposed system is feasible and suitable for implementation.

2.5.1 TECHNICAL FEASIBILITY

The proposed system is technically feasible as it uses well-established technologies such as Python and the Django framework, which are widely supported and suitable for web application development. These technologies provide built-in features for security, database management, and user authentication, making development more efficient.

The development tools required, such as Visual Studio Code, are easily available and simple to use. The system can be developed and executed on standard hardware with basic specifications, without the need for high-end infrastructure.

The platform supports smooth integration between different modules like user, admin, veterinarian, and care centre, ensuring proper functionality. It also allows easy maintenance, updates, and future enhancements if needed.

Moreover, the system can handle multiple users and requests efficiently, ensuring reliable performance. Since all the required technical resources and skills are accessible, the development and implementation of the system are practical and achievable.

Overall, the technical resources, tools, and infrastructure required for the project are readily available, making the system feasible to develop and deploy.

2.5.2 ECONOMIC FEASIBILITY

The proposed system is economically feasible as it requires minimal investment for development and implementation. The project is developed using open-source technologies such as Python and the Django framework, which eliminates licensing costs.
The hardware requirements are also minimal, as the system can run on standard computers without the need for expensive equipment. This reduces the overall cost of setup and maintenance.
In addition, the cost of development is low since the tools used, such as Visual Studio Code, are freely available. Maintenance and future upgrades can also be carried out at a low cost.
The system helps save time and resources by automating manual processes, reducing paperwork, and improving efficiency. This leads to better utilization of resources and long-term cost benefits.
Overall, the project is cost-effective and provides good value by improving stray animal management without requiring significant financial investment.

2.5.3 OPERATIONAL FEASIBILTY

The proposed system is operationally feasible as it is designed to be simple, user-friendly, and easy to operate. The interface is clear and understandable, allowing users, care centres, and veterinarians to use the system without requiring advanced technical knowledge.

The system supports smooth interaction between all modules, enabling users to report stray animals, request consultations, and adopt pets easily. Care centres and veterinarians can efficiently manage requests, update information, and provide services through the platform.

Minimal training is required for users to understand the system, which makes it easy to adopt in real-world situations. The system also ensures proper data management and quick access to information, improving overall efficiency.

By reducing manual work and improving communication, the system can be effectively integrated into daily operations, making it practical and convenient for all stakeholders.

Overall, the system is reliable, easy to use, and suitable for real-time operation.

2.5.4 SCHEDULE FEASIBILTY

The proposed system is schedule feasible as it can be developed and implemented within a reasonable timeframe. The project follows a structured development approach, including planning, design, development, testing, and deployment phases.
The technologies used, such as Python and the Django framework, support faster development, which helps in completing the project on time. Each module of the system can be developed step by step, making the process more manageable.
Proper time management and planning ensure that all tasks are completed within the specified schedule. Any potential delays can be minimized through effective coordination and monitoring.
Overall, the project can be successfully completed within the given time constraints, making it schedule feasible.

2.6. SYSTEM SPECIFICATION

System specification describes the hardware and software requirements needed to run the Stray Dogs or Cats Management System efficiently. The project is developed using modern web technologies to ensure better performance and reliability. The system requires a computer with a minimum Intel i5 processor, 4 GB RAM, and sufficient hard disk space for smooth functioning. Basic input and output devices such as a keyboard, mouse, and monitor are also required. The software environment includes a Windows operating system, Python programming language, and the Django framework for backend development. For designing the user interface, technologies like HTML, CSS, and JavaScript are used. The development of the system is carried out using Visual Studio Code IDE, which provides an efficient environment for coding and testing.

2.6.1 SOFTWARE SPECIFICATION

A Software specification describes the software requirements needed for the development and execution of the Stray Dogs or Cats Management System. The system is developed using the Python programming language along with the Django framework, which supports efficient backend development and database management. The development process is carried out using Visual Studio Code IDE, which provides a user-friendly coding environment. The application runs on a Windows operating system (Windows 10 or higher) and can be accessed through modern web browsers such as Google Chrome. For designing the user interface, technologies like HTML, CSS, and JavaScript are used to ensure an interactive and responsive experience.

Front end

Hypertext Mark-up Language

It is the standard mark-up language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript. Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document. HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page.

HTML provides a means to create structured documents by denoting structural semantics for text such as headings, paragraphs, lists, links, quotes and other items. HTML elements are delineated by tags, written using angle brackets. Tags such as and directly introduce content into the page. Other tags such as surround and provide information about document text and may include other tags as sub elements. Browsers do not display the HTML tags but use them to interpret the content of the page.

Cascading Style Sheets (CSS)

It is a style sheet language used to describe the presentation of a document written in HTML or XML (including XML dialects such as SVG, Math or XHTML). CSS describes how elements should be rendered on screen, on paper, in speech, or on other media.

JavaScript

JavaScript is a scripting language used to create interactive elements in web applications. It is used for form validation, dynamic content updates, and improving user experience.

Back end

Python

Python is a high-level, interpreted, general-purpose programming language that emphasizes code readability and simplicity. It was developed by Guido van Rossum and first released in 1991.

Stray dogs and cats

Python allows developers to write programs using fewer lines of code compared to many other programming languages.

Features:

- Python uses a clean and readable syntax similar to the English language.
- This makes it easy for beginners to learn and understand programming concepts quickly.
- Python is an interpreted language, meaning the code is executed line-by-line.
- This makes debugging easier and eliminates the need for a separate compilation step.
- Python supports object-oriented concepts such as:

  o Classes and objects
  o Inheritance
  o Polymorphism
  o Encapsulation

- This helps in building reusable and modular code.
- Python programs can run on different operating systems like Windows, Linux, and macOS without modification. This is known as portability.

Advantages :

- Python is one of the easiest programming languages to learn due to its simple syntax and readability. This reduces the learning curve for beginners.
- Developers can write programs faster because Python requires fewer lines of code compared to languages like Java or C++.
- Python can be used for a wide range of applications, including:

  o Web development (Django, Flask)
  o Data analysis (Pandas, NumPy)
  o Machine learning (TensorFlow, Scikit-learn)
  o Automation and scripting

- Python provides numerous libraries and frameworks that simplify complex tasks, reducing development time.
- Python programs can run on multiple platforms without modification, making it highly portable.

Disadvantages:

- Python is slower than compiled languages like C and C++ because it is interpreted. This can be a limitation for performance-critical applications.
- Python uses more memory compared to other programming languages, making it less suitable for memory-intensive applications.
- Python is not widely used for mobile app development compared to languages like Java (Android) or Swift (iOS)
- Due to dynamic typing, errors may occur during runtime if variables are used incorrectly, which can lead to unexpected crashes.
- Python has limitations in multi-threading (due to Global Interpreter Lock), which affects performance in CPU-bound tasks.

In conclusion, Python is a powerful and versatile programming language that is widely used in modern software development. Its simplicity, readability, and vast library support make it an ideal choice for beginners as well as professionals. However, it has certain limitations in terms of performance and memory usage. Despite these drawbacks, Python continues to be one of the most popular programming languages in the world.

Django Framework

Django is a Python-based web framework that follows the Model-View-Template (MVT) architecture. It provides built-in security features, rapid development tools, and efficient database management.

Features of Django:

1. Built-in authentication system
2. Strong security protection against SQL injection and CSRF attacks
3. Fast development and clean design
4. Scalable and maintainable architecture

Database

SQLite

SQLite is a lightweight, open-source relational database management system that is designed to be simple, fast, and self-contained. Unlike traditional database systems such as MySQL or PostgreSQL, SQLite does not require a separate server process to operate. Instead, it stores the entire database in a single file on the local system, making it highly portable and easy to use. It uses Structured Query Language (SQL) for managing and querying data, allowing users to perform operations like creating tables, inserting records, updating data, and retrieving information efficiently. One of the main advantages of SQLite is its simplicity and minimal setup,

as it does not need installation or configuration. It is widely used in embedded systems, mobile applications, and small-scale applications where a full-fledged database server is not necessary.

Operating System

Windows

The system runs on Windows 8 or above. The operating system supports necessary development tools, server configuration, and browser compatibility. It provides a stable and user-friendly environment for development and deployment.

Software Tools Used

- IDE: Visual art studio
- Web Browser: Google Chrome / Internet Explorer
- Web Server: Django Server Technology Stack Summary

Front End : HTML, CSS, JavaScript

Back End : Python

Framework: Django

Database : SQLite

Operating System : Windows 8 or above

2.6.2 TOOLS SPECIFICATION

The following tools are used for the development and testing of the Fixora system:

Tools Used:

- Google Chrome
- Internet Explorer
- Visual Art Studio (Development IDE)
- SQLite (Database Management Tool)

Platform:

- Windows Operating System

2.6.3 HARDWARE SPECIFICATION

- Processor: Intel i5 or above
- RAM: 4 GB or above
- Hard Disk: 100 GB or above
- Display Type: PC Display / Monitor
- Input Devices: Keyboard, Mouse
- Output Devices: Monitor / Mobile Display

3.SYSTEM DESIGN

3.1 INTRODUCTION

System design is an important phase in software development where the overall structure and architecture of the system are planned. It defines how the system will function, how different modules will interact, and how data will flow within the system.

In the **Stray Dogs or Cats Management System**, system design focuses on organizing the application into different modules such as admin, user, veterinarian, and care centre. Each module is designed to perform specific functions, ensuring smooth operation and proper coordination between all components.

The design phase includes defining input and output processes, database structure, and user interfaces. It ensures that the system is efficient, reliable, and easy to use. A well-designed system helps in reducing complexity, improving performance, and making future modifications easier.

Overall, system design provides a clear blueprint for the development of the project and ensures that all requirements are properly implemented.

3.2 INPUT DESIGN

Input design focuses on the way data is collected and entered into the system. It aims to make data entry simple, accurate, and efficient while reducing errors. A well-designed input system ensures that only valid and necessary data is provided by users.

In the **Stray Dogs or Cats Management System**, various input forms are used such as user registration, login, reporting stray animals, adoption requests, and veterinary consultation requests. Each form is designed with clearly labeled fields so that users can easily understand what information is required.

The system uses validation techniques such as required fields, format checking (like email and phone number), and input restrictions to ensure data accuracy. Error messages are displayed when incorrect data is entered, helping users to correct mistakes immediately.

The design also focuses on minimizing the number of input fields to avoid complexity and reduce user effort. Drop-down lists, radio buttons, and checkboxes are used wherever possible to make input faster and more user-friendly.

Security is also considered in input design by protecting sensitive data such as login credentials. Proper authentication and validation methods are used to prevent unauthorized access and incorrect data entry.

Overall, the input design ensures that data entered into the system is accurate, consistent, and easy to manage, which improves the overall efficiency and reliability of the system.

3.3 OUTPUT DESIGN

Output design is concerned with how the system presents processed information to the users in a clear and understandable format. It ensures that the output is meaningful, accurate, and easy to interpret, helping users make informed decisions.
In the **Stray Dogs or Cats Management System**, outputs are displayed through web pages and system notifications. The system provides various outputs such as the list of stray dogs and cats available for adoption, details of reported animals, status updates of requests, and appointment schedules with veterinarians.
The output screens are designed with proper headings, structured layouts, and clear formatting to improve readability. Important information is highlighted so that users can easily understand the results without confusion. The system also ensures that outputs are updated in real-time or at regular intervals to provide the latest information.
Additionally, the system maintains accuracy and consistency in all outputs, reducing the chances of misinterpretation. It also allows users to track their activities and monitor progress, which increases transparency and user satisfaction.

3.4 DATABASE DESIGN

Database design, A most important part of the system design phase. In a database environment, data available are used by several users instead of each program managing its own data, authorized users share data across application with the database software managing the data as an entity.

Primary key is one of the candidate keys that are chosen to be the identifying key for the entire table.

Normalization is a process of organizing the data in database to avoid data redundancy, insertion anomaly, update anomaly & deletion anomaly.

Here are the most commonly used normal forms:

- First normal form (INF)
- Second normal form(2NF)
- Third normal form(3Nf)
- Boyce& Codd normal form (BCNE)

FIRST NORMAL FORM (INF)

As per the rule of first normal form, an attribute (column) of a table cannot hold multiple values. It should hold only atomic values.

SECOND NORMAL FORM (2NF)

A table is said to be in 2NF if both the following conditions hold:

- Table is in INF (First normal form)

- No non-prime attribute is dependent on the proper subset of any candidate key of table.

THIRD NORMAL FORM (3NF)

Dependency of non-prime attribute on any super key should be removed An attribute that is not part of any candidate key is known as non-prime attribute. In other words, 3NF can be explained like this: A table is in 3NF if it is in 2NF and for each functional dependency X-> Y at least one of the following conditions hold:

- X is a super key of table
- Y is a prime attribute of table
- An attribute that is a part of one of the candidate keys is known as prime attribute.

BOYCE CODD NORMAL FORM (BCNF)

It is an advance version of 3NF that's why it is also referred as 3.5NF. BCNF is stricter than 3NF. A table complies with BCNF if it is in 3NF and for every functional dependency X->Y, X should be the super key of the table.

3.5.DATA FLOW DIAGRAM (DFD)

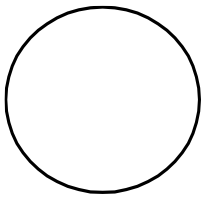
The Data flow diagram (DFD) also known as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of the input data to the system, various processing carried out on these data and the output data generated by the system. The main reason why this DFD technique is so popular is probably because of the fact that DFD iPro's a very simple formalism. It is simple to understand and use. A DFD model uses a very limited number of primitive symbols to represent the functions performed by a system and the data flow among these systems. Starting with a set of high-level functions that a system performance of DFD model in hierarchically it represents various sub functions. The Data Flow Diagramming technique also follows a simple set of intuitive concepts and rules. Data flow diagram (DFD) is used to show how data flows through the system and the processes that transform the input data into output. Data flow diagrams are a way of expressing system requirements in a graphical manner. DFD represents one of the most ingenious tools used for structured analysis.

BASIC DFD SYMBOLS

External Entity Symbol:

An external entity such as a user, project manager etc. is represented by a rectangle. The external entities are essentially those physical entities external to the application system, which interact with the system by inputting data to the system or by consuming the data produced by the system.

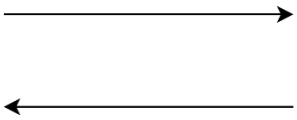
Process Symbol:



The functions are using circles. Bubbles are annotated with the names of the corresponding functions. They convert data into information.

Data Flow Symbol:

DATA FLOW



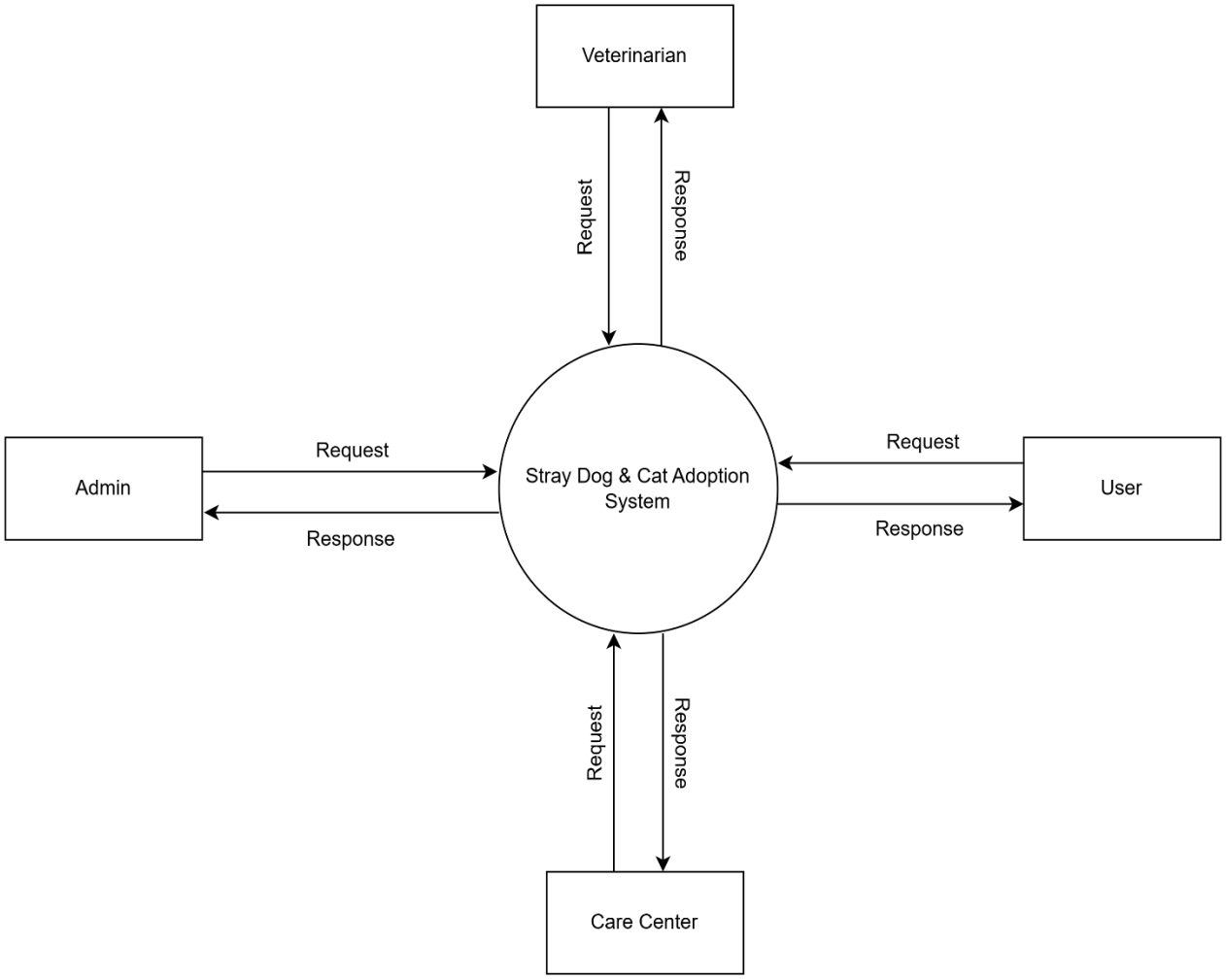
A directed arc or an arrow is used as a Data Flow Symbol. This represents the data flow occurring between two processes or between an external entity and a process: in direction of the Data Flow Arrow. Data flow Symbols are annotated with corresponding data names.

Data Store Symbol:

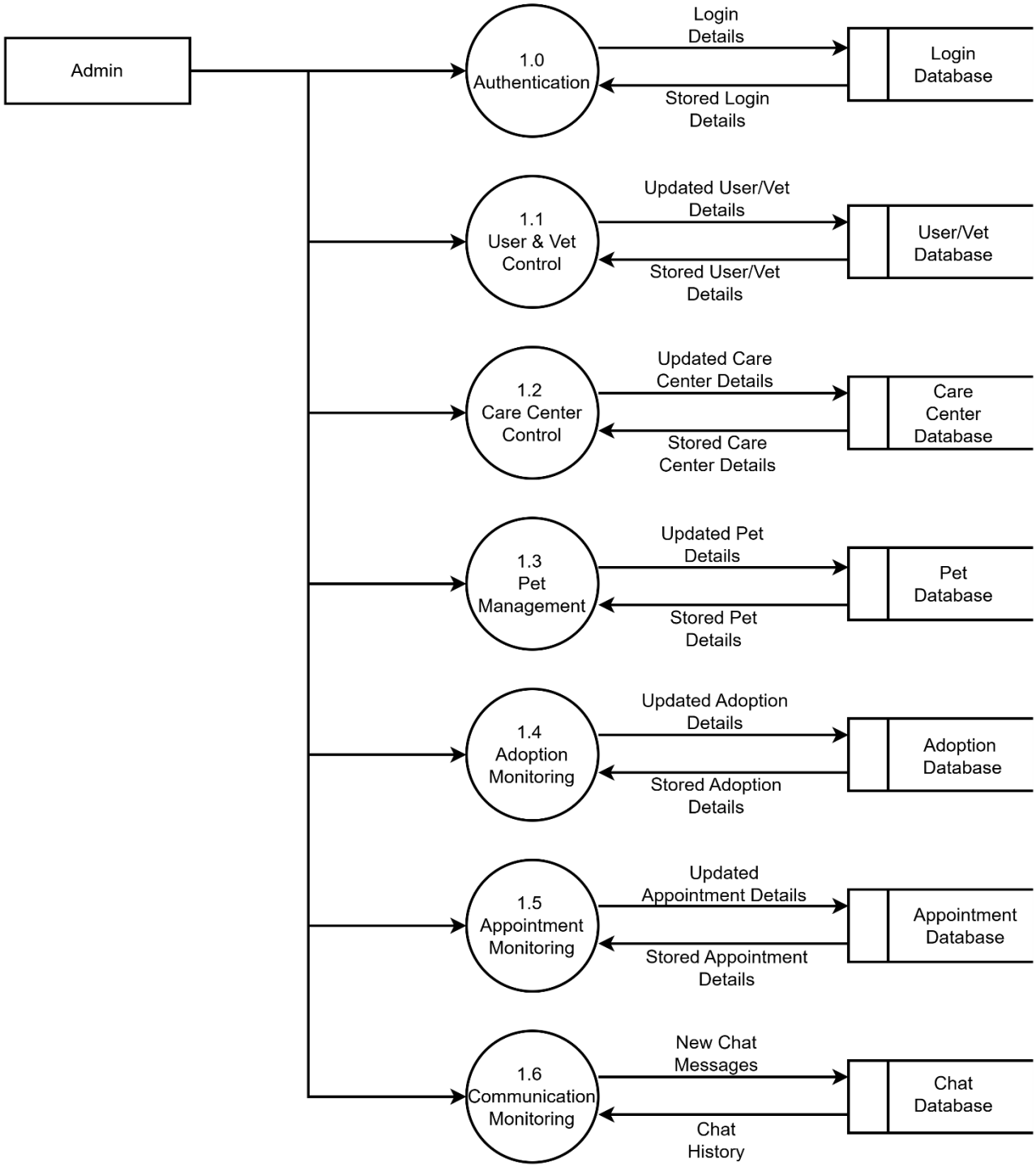
A Data Store represents a logical file; it is represented using two parallel lines. A logical file can represent either Data Store Symbol, which can represent either data structure or a physical file on disk. Each data store is connected to a process by means of a Data Flow Symbol.

DFD FOR STRAY DOGS AND CATS

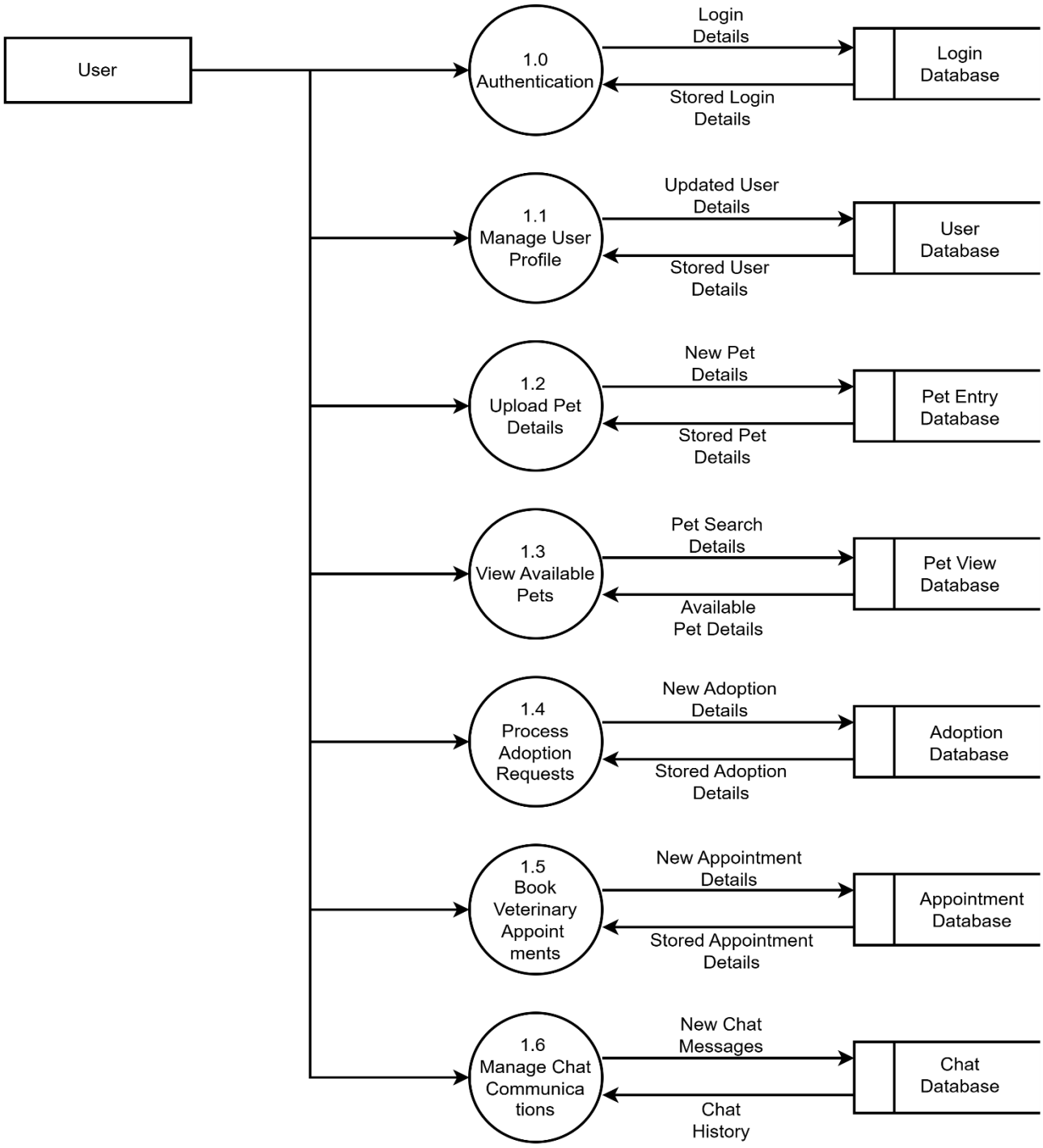
LEVEL 0



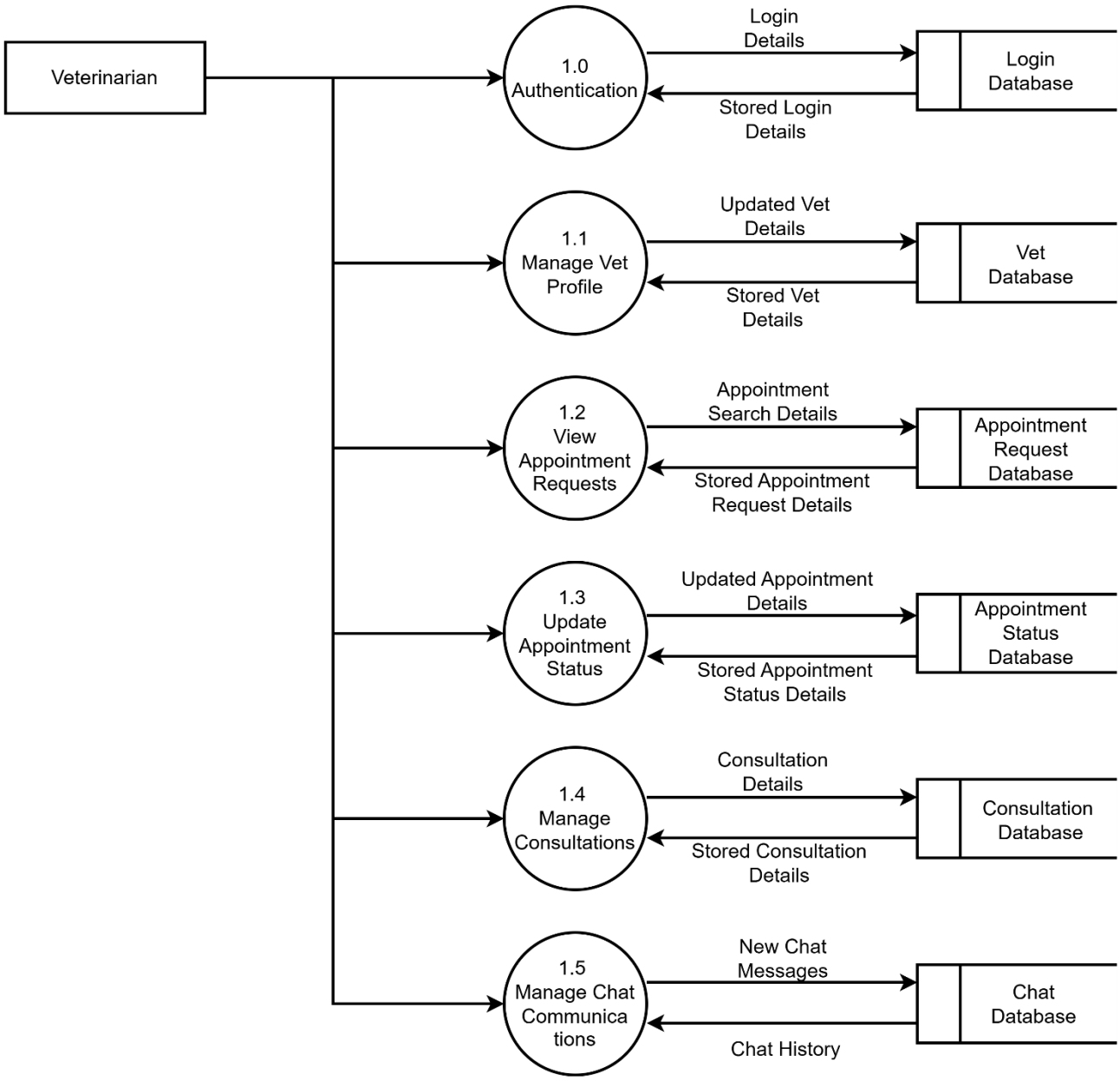
LEVEL 1 ADMIN



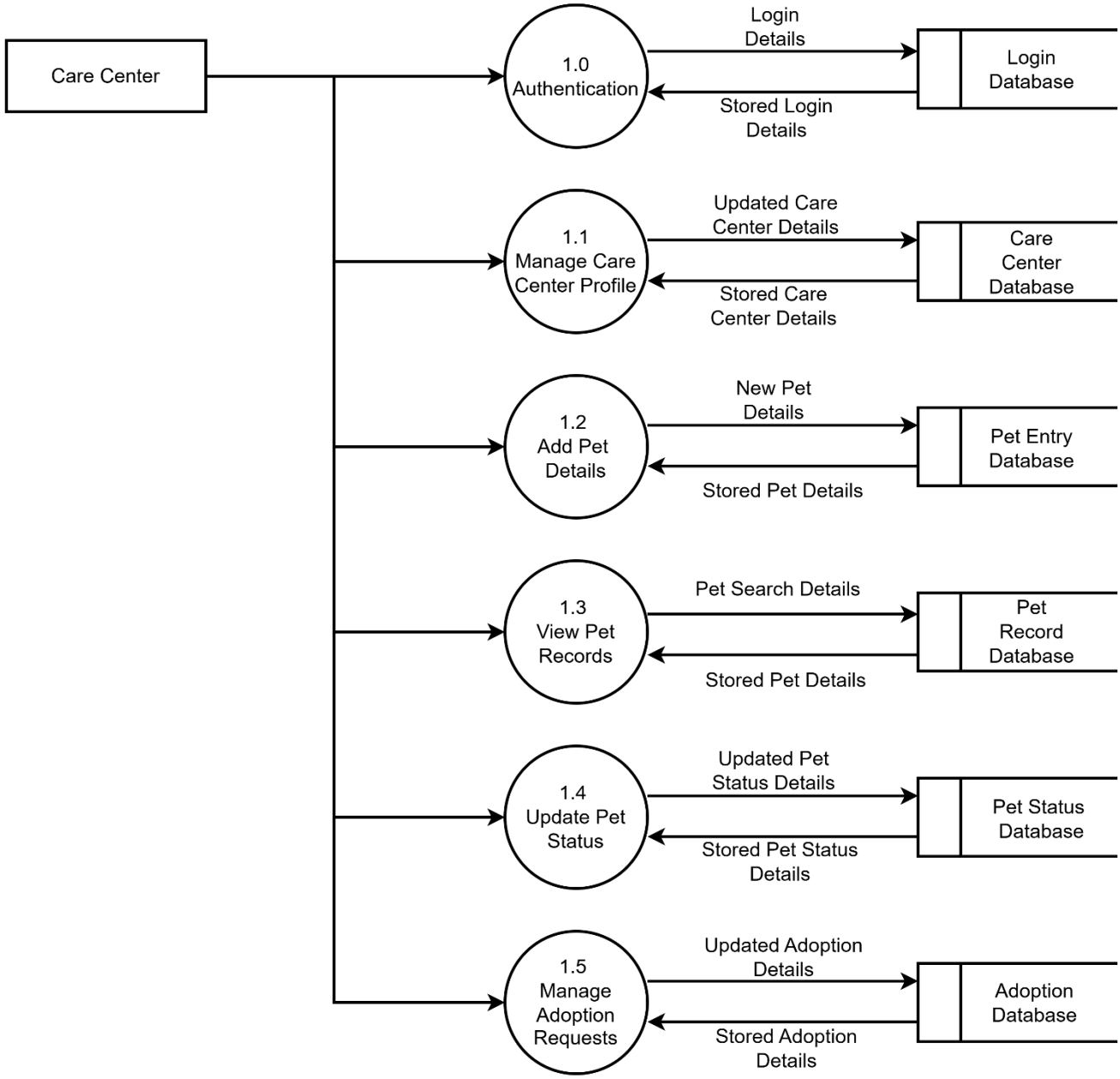
LEVEL 1 USER


User
1.0 Authentication
Login Details
Stored Login Details
Login Database
1.1 Manage User Profile
Updated User Details
Stored User Details
User Database
1.2 Upload Pet Details
New Pet Details
Stored Pet Details
Pet Entry Database
1.3 View Available Pets
Pet Search Details
Available Pet Details
Pet View Database
1.4 Process Adoption Requests
New Adoption Details
Stored Adoption Details
Adoption Database
1.5 Book Veterinary Appoint ments
New Appointment Details
Stored Appointment Details
Appointment Database
1.6 Manage Chat Communica tions
New Chat Messages
Chat History
Chat Database

LEVEL 1 VETERINARIAN



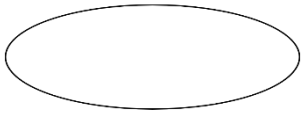
LEVEL 1 CARE CENTER



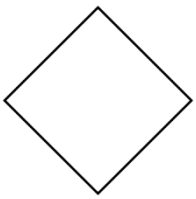
3.6 ENTITY RELATIONSHIP DIAGRAM

ER Diagram stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships. ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes and diamond shapes to represent relationships.

Attributes: Attributes are the properties which define the entity type. In ER diagram, attribute is represented by an oval.

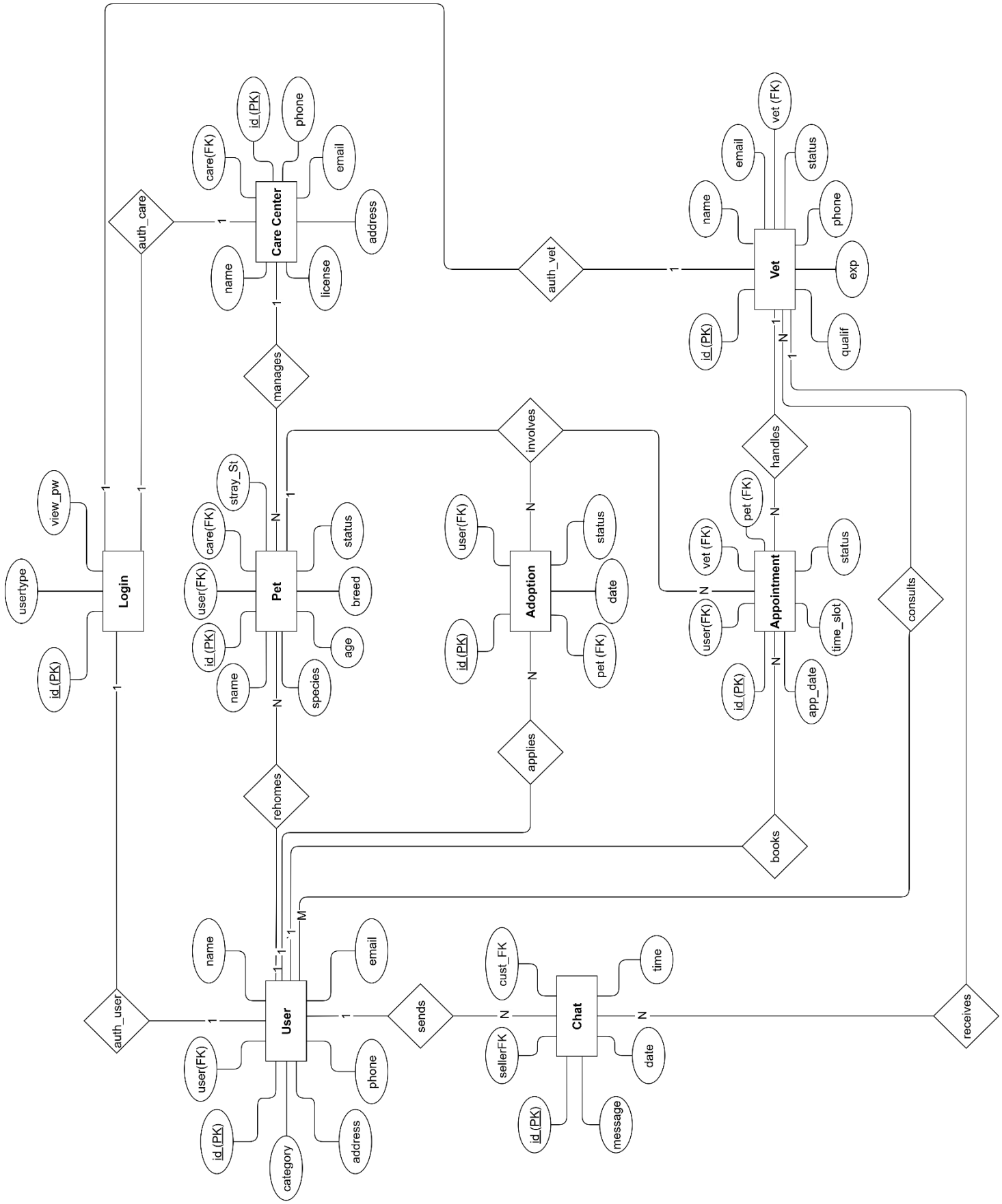


Relationship: A relationship type represents the association between entity types. In ER diagram, relationship type is represented by a diamond.



Entity: An entity is an object or component of data. An entity is represented as rectangle in an ER diagram.

ER DIAGRAM

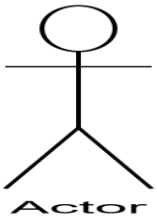

Login
id (PK)
usertype
view_pw
auth_care
auth_user
auth_vet
Care Center
id (PK)
name
license
address
email
phone
care(FK)
manages
Pet
id (PK)
name
user(FK)
care(FK)
stray_St
species
breed
age
status
rehomes
Adoption
id (PK)
user(FK)
pet (FK)
date
status
involves
applies
Vet
id (PK)
name
email
phone
exp
qualif
status
vet (FK)
handles
Appointment
id (PK)
user(FK)
vet (FK)
pet (FK)
app_date
time_slot
status
books
consults
User
id (PK)
user(FK)
name
category
address
phone
email
sends
Chat
id (PK)
sellerFK
cust_FK
message
date
time
receives

3.7.USE CASE DIAGRAM

A use case diagram is a graphical depiction of a user's possible interactions with a system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses.

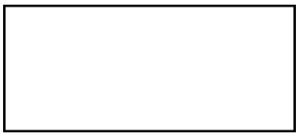
Actor:

The users that interact with a system. An actor can be a person, an organization, or an outside system that interacts with your application or system. They must be external objects that produce or consume data.



System:

A box that sets a system scope to use cases. All use cases outside the box would be considered outside the scope of that system.

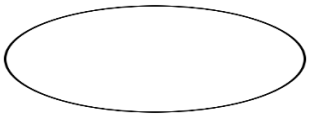


Association:

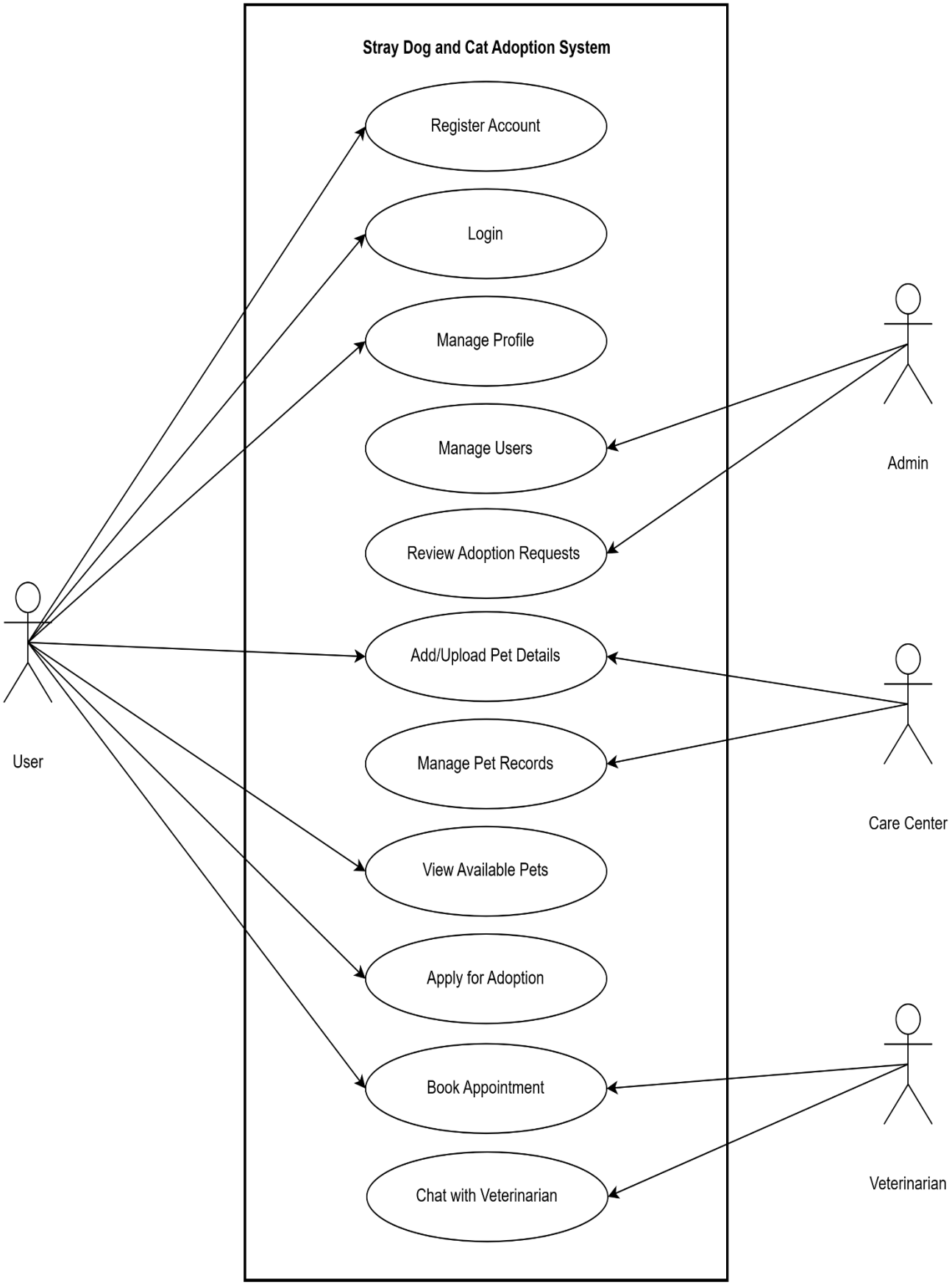
A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

Use cases:

Horizontally shaped oval that represent the different uses that the system might have.



USECASE DIAGRAM



4.SYSTEM DEVELOPMENT

4.1 MODULE DESCRIPTION

The **Stray Dogs or Cats Management System** is divided into different modules to ensure smooth functioning and better organization of the system. Each module performs specific tasks and works together to achieve the overall objectives of the project.

Modules:

- Admin
- User
- Veterinarian
- Care Centre

1. Admin Module

The admin module is responsible for managing the entire system. The admin can view and manage users, care centres, and veterinarians. It ensures proper monitoring and control of all activities in the system, maintaining security and reliability.

Functions:

- Admin Login
- Manage Users
- Manage Care Centres
- Manage Veterinarians
- Monitor System Activities
- Ensure Data Security
- Maintain System Efficiency

2. User Module

The user module allows individuals to register and log in to the system. Users can view stray dogs and cats available for adoption, adopt pets, report stray animals, request veterinary consultations, and track appointment status

Functions:

- User Registration
- User Login
- View Stray Animals
- Adopt Pets

Stray dogs and cats

- Report Stray Animals
- Report Veterinary Consultation
- View Appointments
- Track Status

3. Veterinarian Module

The veterinarian module enables registered veterinarians to manage consultation requests. They can view user requests, schedule appointments, provide medical assistance, and update their profile details.

Functions:

- Registration
- Login
- View Requests
- Schedule Appointments
- Provide Medical Assistance
- Update Profile
- Manage Appointments

4. Care Centre Module

The care centre module plays a key role in managing stray animals. Care centres can add pets for adoption, view and manage animal details, handle user requests, and coordinate with veterinarians. They also update the status of animals in the system.

Function:

- View Veterinarians
- View Requests
- Add Pets for Adoption
- Manage Stray Animals
- Update Status
- Coordinate with Veterinarians
- Maintain Records

5.SYSTEM IMPLEMENTATION

5.1 TESTING

Testing is an important phase in system development that ensures the system functions correctly and meets all specified requirements. It helps in identifying and fixing errors, ensuring reliability and performance of the application.

In the **Stray Dogs or Cats Management System**, testing is carried out at different stages of development. Each module such as admin, user, veterinarian, and care centre is tested individually to ensure proper functionality. After module testing, integration testing is performed to verify that all modules work together smoothly.

1. Unit Testing

Unit testing is the process of testing individual modules of the system separately to ensure they work correctly. In this system, each module like admin, user, veterinarian, and care centre is tested individually.

It helps in identifying errors early and ensures that each function performs as expected.

2. Integration Testing

Integration testing is the process of testing combined modules to ensure they work together correctly. In this system, modules like admin, user, veterinarian, and care centre are integrated and tested.

It helps in identifying issues in data flow and communication between modules, ensuring smooth system operation.

3. System Testing

System testing is done on the complete Stray Dogs and Cats Management System after integration to ensure that the system works according to requirements. It verifies overall functionality, data flow between modules, and checks both functional and non-functional aspects such as performance, security, and usability. Any errors identified during testing are corrected before final deployment.

4. Acceptance Testing

Acceptance testing is performed to check whether the Stray Dogs and Cats Management System meets the user requirements and is ready for deployment. It is carried out from the end-user perspective to ensure that all functionalities such as reporting stray animals, adoption process, and medical updates work correctly in real-time usage. The system is accepted only after satisfying all requirements and user expectations.

5. Regression Testing

Regression testing is performed to ensure that recent changes or updates in the Stray Dogs and Cats Management System do not affect the existing functionality. It is carried out after bug fixes or enhancements to verify that previously working features such as reporting, adoption, and medical updates still function correctly.

5.2 VALIDATION CHECK

Validation check in the Stray Dogs and Cats Management System ensures that all data entered by users is correct, complete, and meaningful before it is stored in the database. It prevents invalid inputs during registration, animal reporting, and adoption processes. The system uses checks such as required field validation to ensure no important field is left empty, format validation for correct email, phone number, and date formats, range validation for numeric limits, and consistency validation to maintain proper and logical data. These checks help improve data accuracy, reduce errors, and ensure smooth system operation.

5.3 SYSTEM IMPLEMENTATION

System implementation is the phase where the Stray Dogs and Cats Management System is deployed and made fully operational after successful testing. It involves converting the designed system into a working system in the real environment.

The implementation process includes installing the system on a web server, configuring the database, and integrating all modules such as Admin, User, Veterinarian, and Care Centre. The backend and frontend components are connected and tested in the live environment to ensure proper functionality.

During implementation, user accounts are created and role-based access control is applied so that each user can access only their permitted features. The system is also checked for performance and security issues before full deployment.

After deployment, the system is monitored regularly to ensure smooth operation. Necessary updates, bug fixes, and database backups are performed as part of maintenance to improve reliability and efficiency.

5.4 SECURITY

Security is a key feature of the Stray Dogs and Cats Management System to protect user data and prevent unauthorized access. The system uses login authentication to ensure that only valid users can access the application. Role-based access control is implemented so that different users such as admin, user, veterinarian, and care centre staff can access only their permitted modules.

In addition, strong password policies are followed to encourage users to create secure passwords. Session management is used to automatically log out inactive users, reducing the risk of unauthorized access. Sensitive data such as user credentials is protected using encryption techniques to ensure data safety.

6.SYSTEM MAINTENANCE AND FUTURE ENHANCEMENT

6.1.SYSTEM MAINTENANCE

System maintenance for the stray dogs and cats management system involves continuous monitoring and updating of the application to ensure its smooth and efficient functioning. After deployment, regular maintenance is required to fix any errors or bugs that may occur during real-time usage. It also includes updating the database with new information about stray animals, rescue operations, and adoption records.

The system will be periodically checked for performance issues such as slow response time or data inconsistencies. Security updates will also be applied to protect user data and prevent unauthorized access. Backup of important data will be taken regularly to avoid data loss in case of system failure.

Proper maintenance ensures that the system remains reliable, accurate, and useful for users like animal rescuers, care centers, and administrators in managing stray dogs and cats effectively.

6.2.FUTURE ENHANCEMENT

The Stray Dogs and Cats Management System can be improved in the future by adding new features and advanced technologies to make it more efficient and user-friendly. One major enhancement can be the development of a mobile application so that users can report stray animals easily using their smartphones.

Artificial Intelligence (AI) can be integrated to identify animal types, predict health conditions, and suggest treatment based on symptoms. GPS tracking can also be added to locate rescued animals and nearby care centres more accurately.

The system can be expanded to include online adoption services, allowing users to view available animals and complete adoption procedures digitally. Notifications and alert systems can be improved to inform users about rescue updates, vaccination schedules, and adoption status.

In the future, cloud storage can be implemented to handle large amounts of data securely and allow easy access from anywhere. These enhancements will improve the overall performance, usability, and effectiveness of the system.

7.CONCLUSION

7.1 CONCLUSION

The Stray Dogs and Cats Management System is a well-structured application developed to support the effective management of stray animal rescue, treatment, and adoption activities. It provides a centralized platform where administrators, veterinarians, care centres, and users can work together in an organized manner to ensure better animal welfare services.

This system helps in reducing the challenges faced in traditional manual methods such as data loss, miscommunication, and delay in rescue operations. By digitizing the entire process, it improves efficiency, accuracy, and transparency in handling stray animal information. Every detail such as animal registration, medical treatment, adoption records, and care centre updates can be managed easily through this system.

The application also encourages social responsibility by allowing more people to participate in animal rescue and adoption activities. It helps in creating awareness about the importance of caring for stray dogs and cats and ensures that they receive proper medical attention and shelter when needed.

In addition, the system provides a reliable database that supports quick decision-making and better coordination among different users. This makes the entire process of animal management more systematic and time-saving.

Overall, the project successfully achieves its objective of providing an efficient and user-friendly solution for stray animal management. With future improvements and technological advancements, it has the potential to become even more effective in supporting animal welfare and rescue operations on a larger scale.

8.APPENDIX

8.1 APPENDIX A. TABLE DESIGNS

1. Login Table

Table name: login
Description: details of login
Primary key: user id

Field	**Data Type**	**Constraints**	**Description**
id	AutoField	Primary Key	Unique user ID
username	CharField	From AbstractUser	Login username
password	CharField	Encrypted (Django)	User password
email	EmailField	Unique (recommended)	Email address
usertype	CharField	Null=True	Defines role (User/Vet/Carecenter/Admin)
view_password	CharField	Null=True (Not recommended)	Displays password (unsafe)

2. User Table

Table name: user
Description: details of user

Field	Data Type	Constraints	Description
id	AutoField	Primary Key	Unique user ID
user	ForeignKey(Login)	CASCADE	Linked Login Account
Name	CharField	Null=True	username
Password	CharField	Null=True(not recommended)	Password stored(unsafe)
email	EmailField	Required	Email ID
Address	CharField	Null=True	Address of User
Phone	CharField	Null=True	Contact Number
Category	CharField	Null=True	User Category
Image	FileField	Null=True	Profile image

3. Vet Table

Table name: Vet
Description: details of vet
Primary key: id

Field	Data Type	Constraints	Description
Id	AutoField	Primary key	Vet ID
Vet	Foreign key(login)	CASCADE	Login Reference
Name	CharField	Null=True	Vet name
Password	CharField	Null=True	Password(not recommended)
Email	EmailField	Required	Email
Qualification	CharField	Null=True	Educational Qualification
Experience	CharField	Null=True	Work Experience
Phone	CharField	Null=True	Contact number
Image	FileField	Null=True	Profile image
Status	CharField	Default='pending'	Approval status

4. Care center Table

Table name: care center
Description: details of care center
Primary key: category id

Field	Data Type	Constraints	Description
Id	AutoField	Primary key	Care center ID
Care	Foreign Key(login)	Null=True	Login reference
Name	CharField	Null=True	Care center name
Password	CharField	Null=True	Password(not recommended)
Email	Emailfield	Required	Email
Address	CharField	Null=True	Location
Phone	CharField	Null=True	Contact number
Image	Filefield	Null=True	Image/logo
CareLicense	CharField	Null=True	License number

5. Pet Table

Table name: pet table
Description: details of pet
Primary key: claim id

Field	Data Type	Constraints	Description
Id	AutoField	Primary key	Pet ID
Care	Foreign key(carecenter)	Null=True	Care center reference
User	Foreign key(user)	Null=True	User reference
Name	CharField	Null=True	Pet name
Species	CharField	Null=True	Dog/Cat
Breed	CharField	Blank=True, Null=True	Breed
Age	CharField	Null=True	Age
Gender	CharField	Null=True	Gender
Description	CharField	Null=True	Details about pet
Img	Imagefield	Null=True	Pet image
Status	CharField	Default='For Adoption'	Adoption status
Stray_status	CharField	Default='Stray_dog&cat'	Stray category

6. Appointment Table

Table name: appointment
Description: details of appointment
Primary key: request id

Field	Data type	Constraints	Description
Id	AutoField	Primary key	Appointment ID
User	ForeignKey(User)	CASCADE	User reference
Vet	ForeignKey(vet)	CASCADE	Vet reference
Pet	ForeignKey(pet)	Null=True	Pet reference
Appointment date	Datefield	Required	Appointment date
Time_slot	CharField	Null=True	Assigned time
Status	CharField	Choices	Pending/scheduled/completed/Rejected

7. Adoption Table

Table name: Adoption
Description: details of adoption
Primary key: report id

Field	**Data type**	**Constraints**	**Description**
Id	AutoField	Primary key	Adoption ID
User	ForeignKey(User)	CASCADE	Adopter
Pet	ForeignKey(pet)	CASCADE	Adopted pet
Date	DateTimeField	Auto now add	Adoption date
Status	CharField	Default='available'	Adoption status

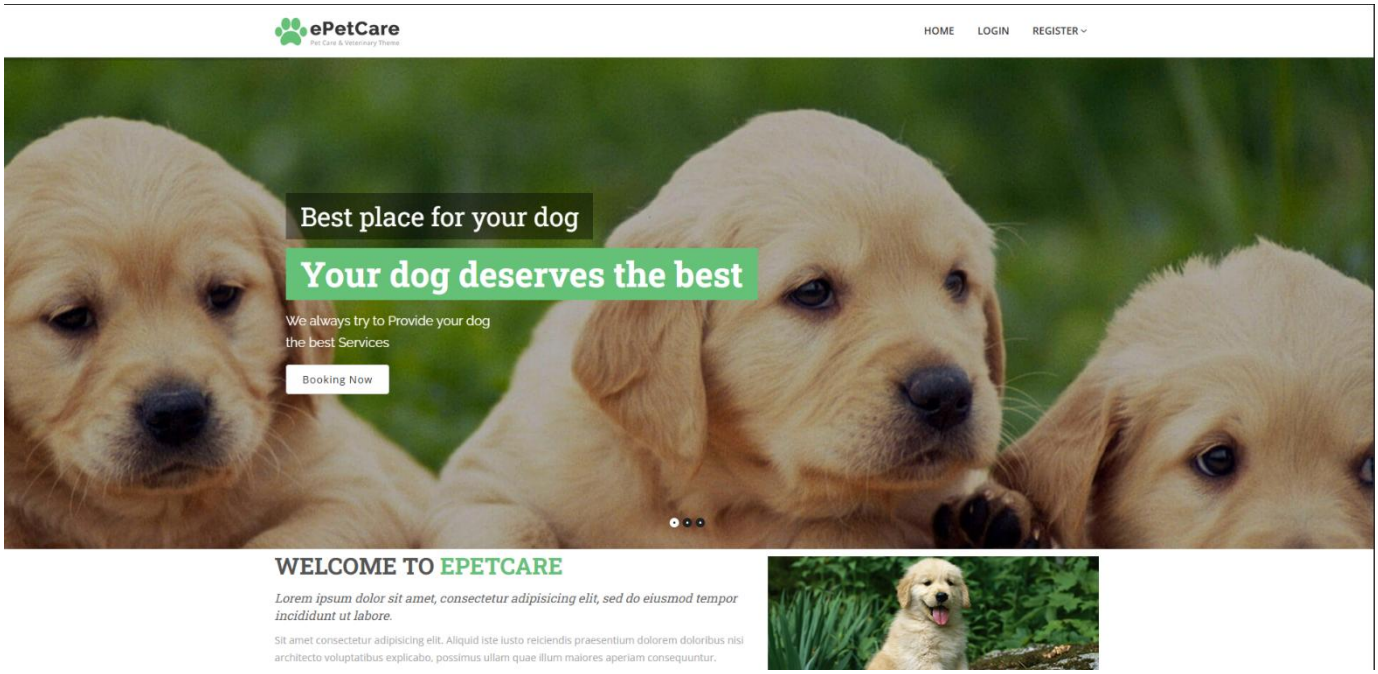
8. Chat Table

Table name: chat
Description: details of chat
Primary key: complaint id

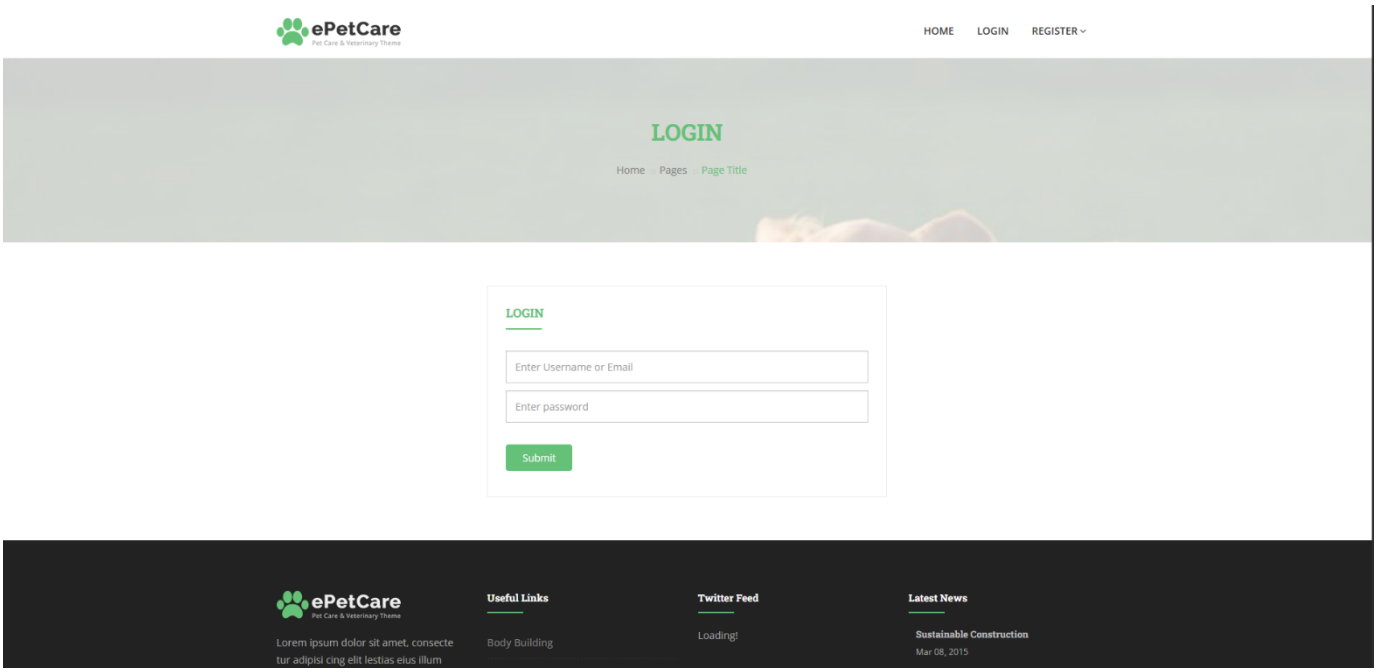
Field	Data Type	Constraints	Description
Id	AutoField	Primary key	Chat ID
Seller id(user)	ForeignKey(User)	CASCADE	Sender
Customer id(vet)	ForeignKey(vet)	CASCADE	Receiver
Message	CharField	Max 300 chars	Chat message
Date	DateField	Auto now	Message date
Time	CharField	Required	Message time
U type	CharField	Required	User type

8.2.APPENDIX B.SAMPLE INPUTS AND OUTPUT DESIGN

Home:

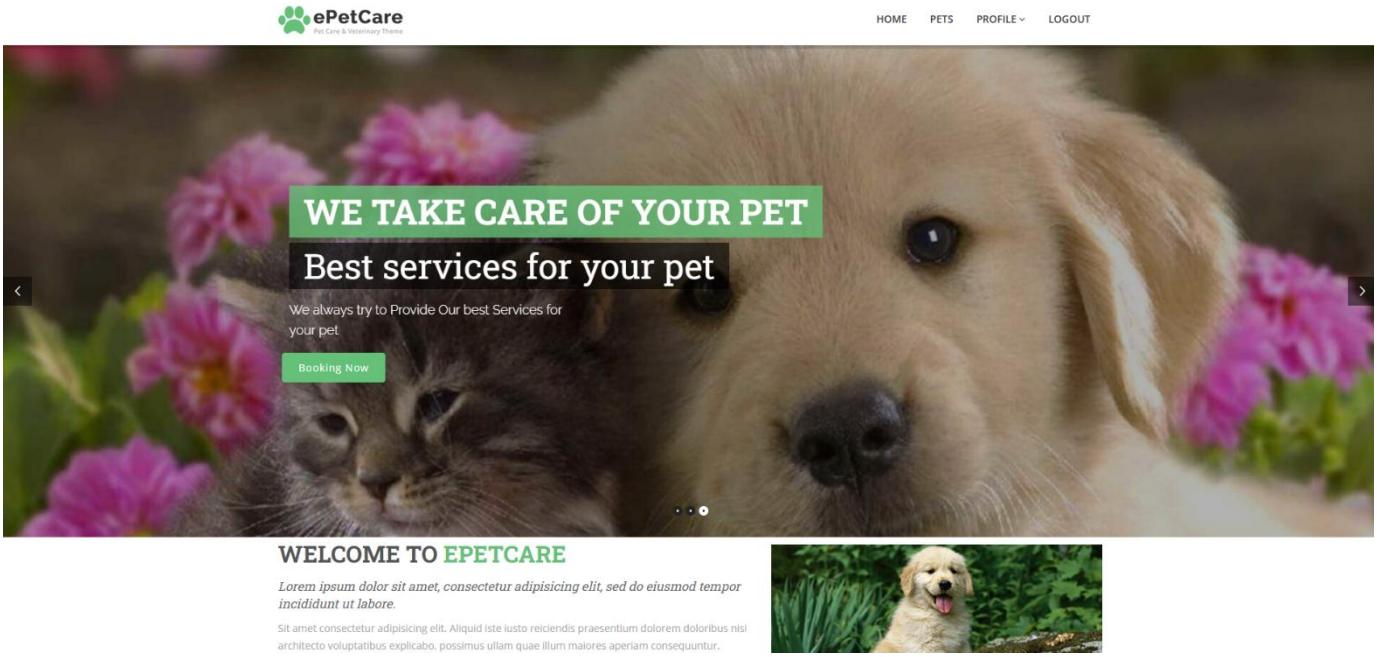


Login:

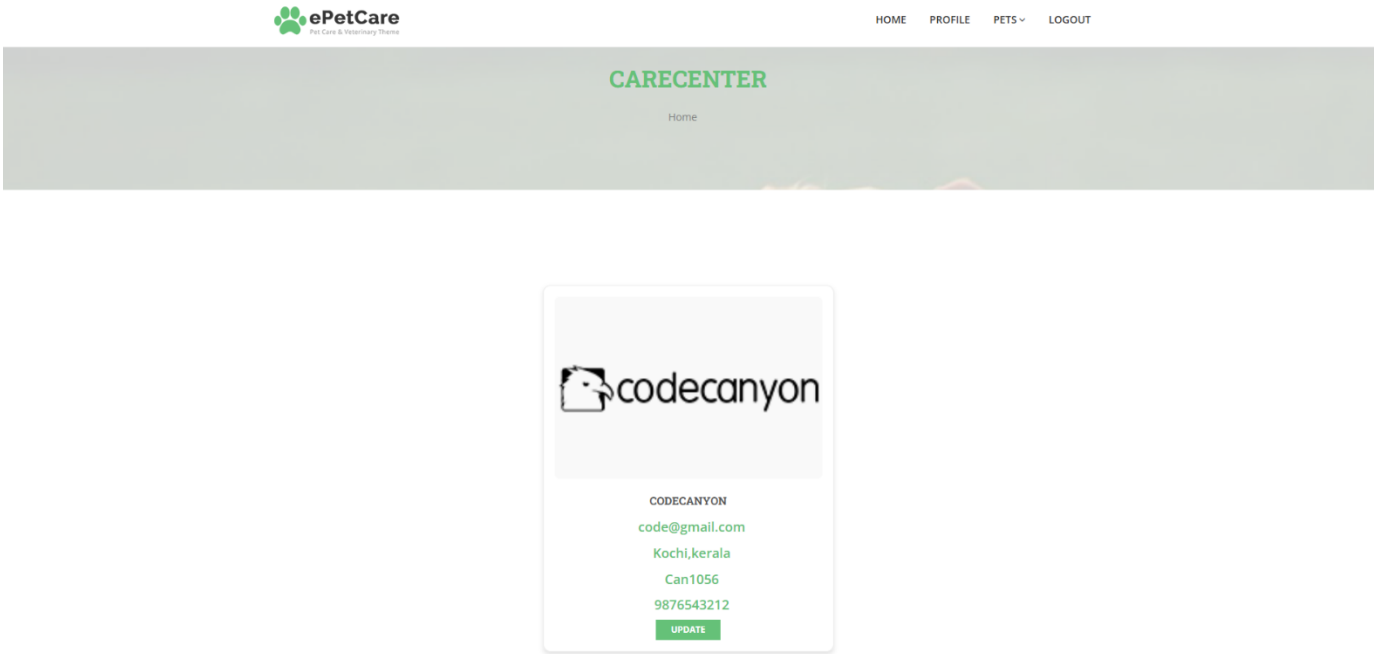


Stray dogs and cats

Admin Dashboard:

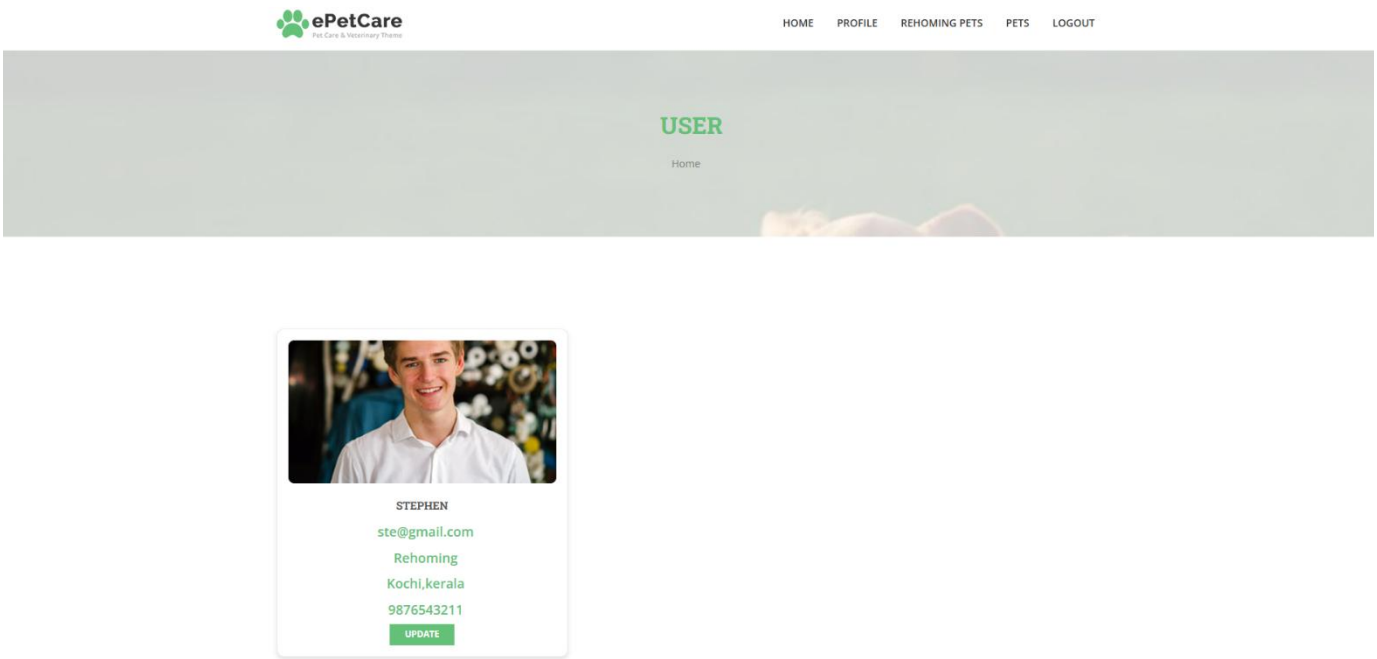


Care Center Login:

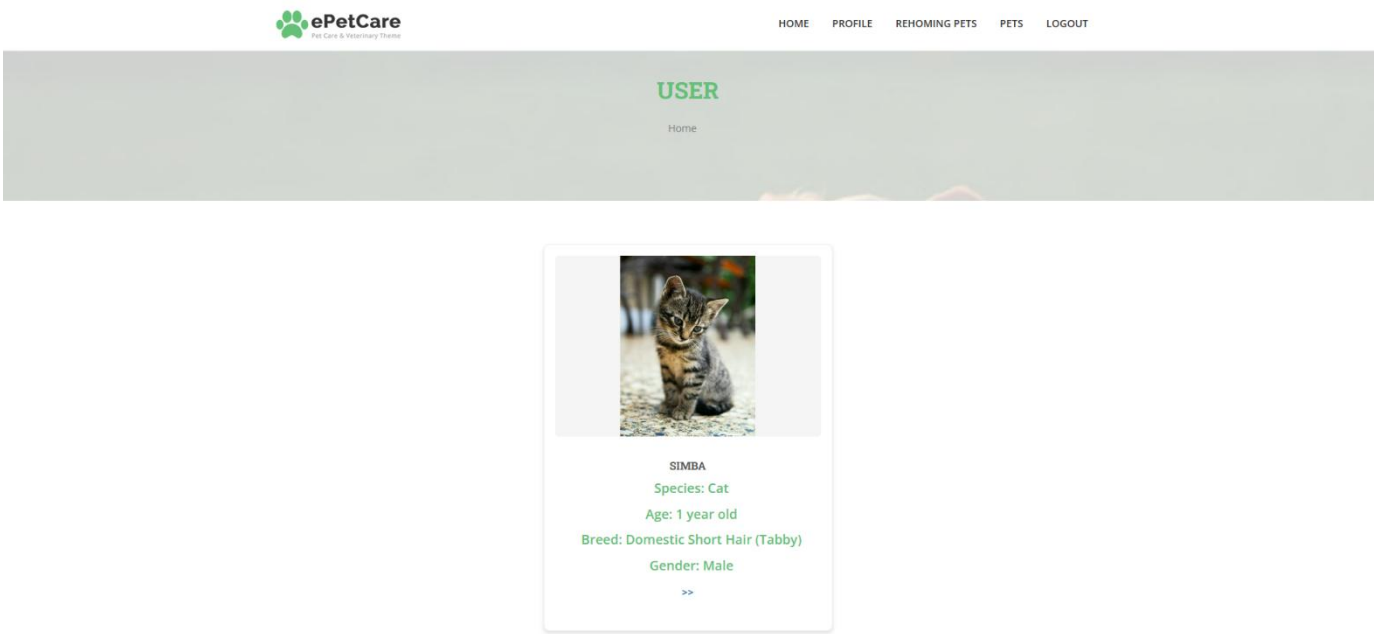


Stray dogs and cats

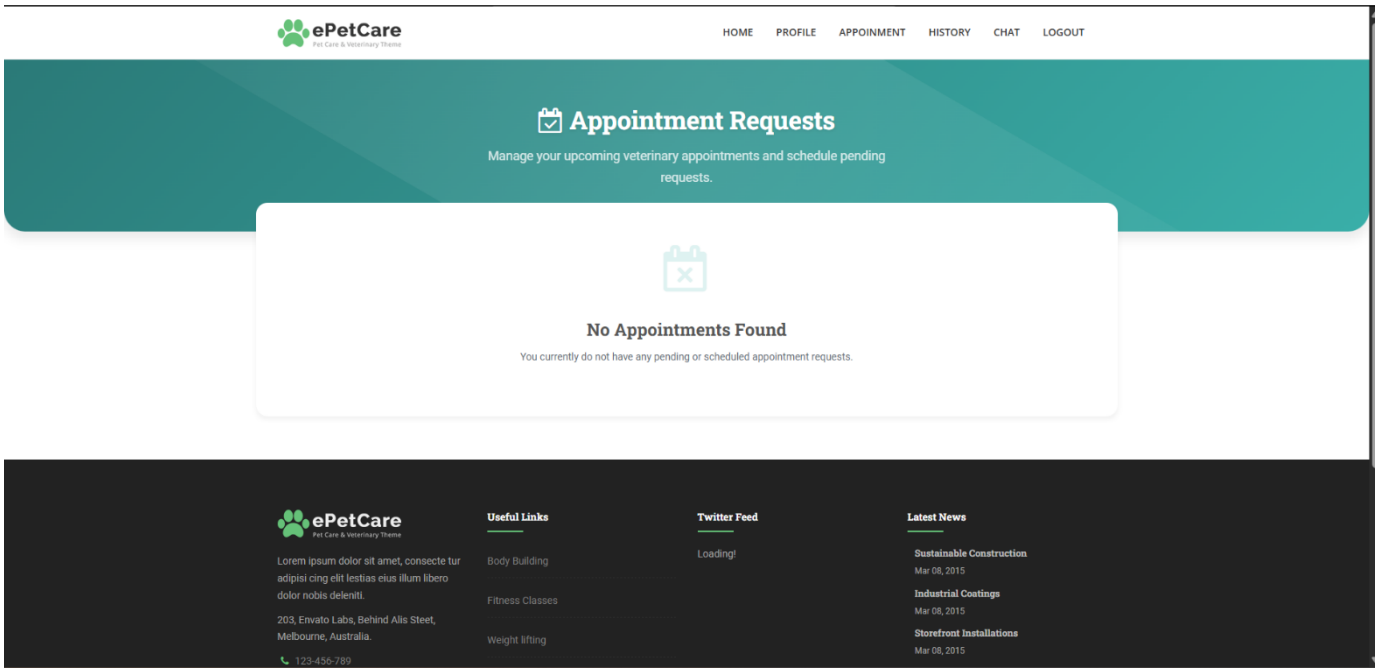
User Home Page:



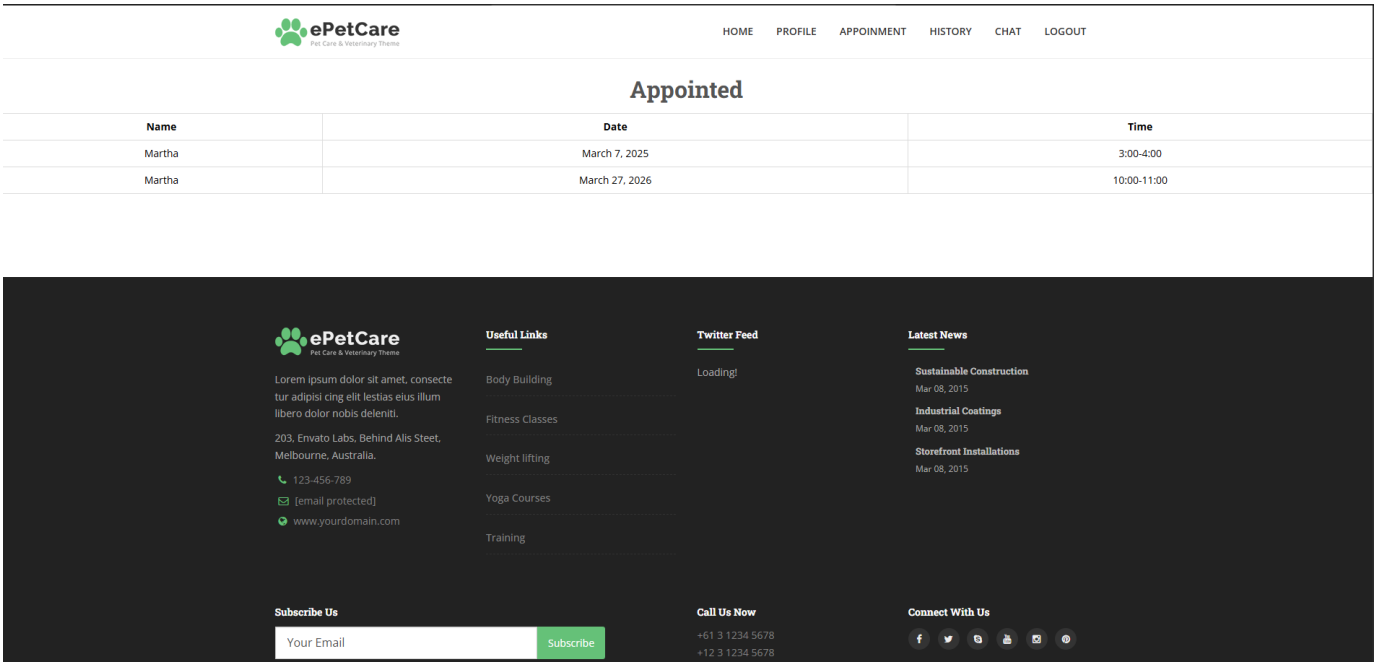
View Pets:



Vet's Appointment Request:

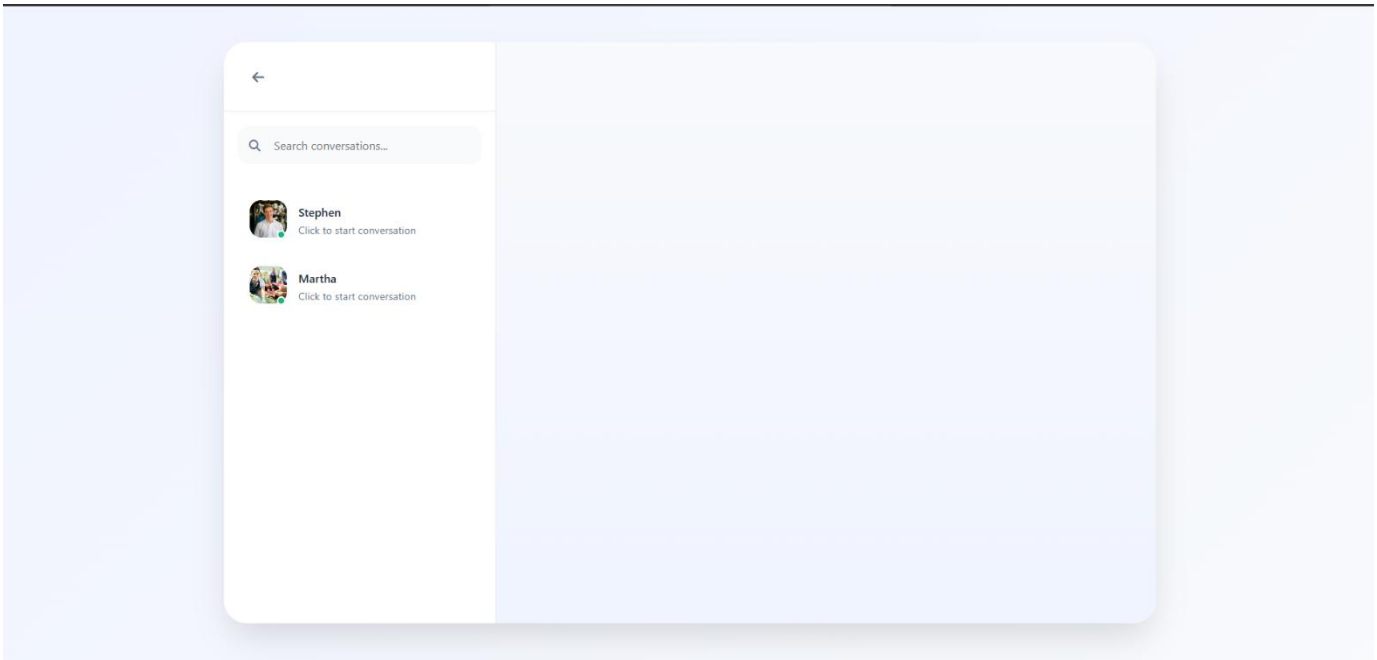


Appointed Page:

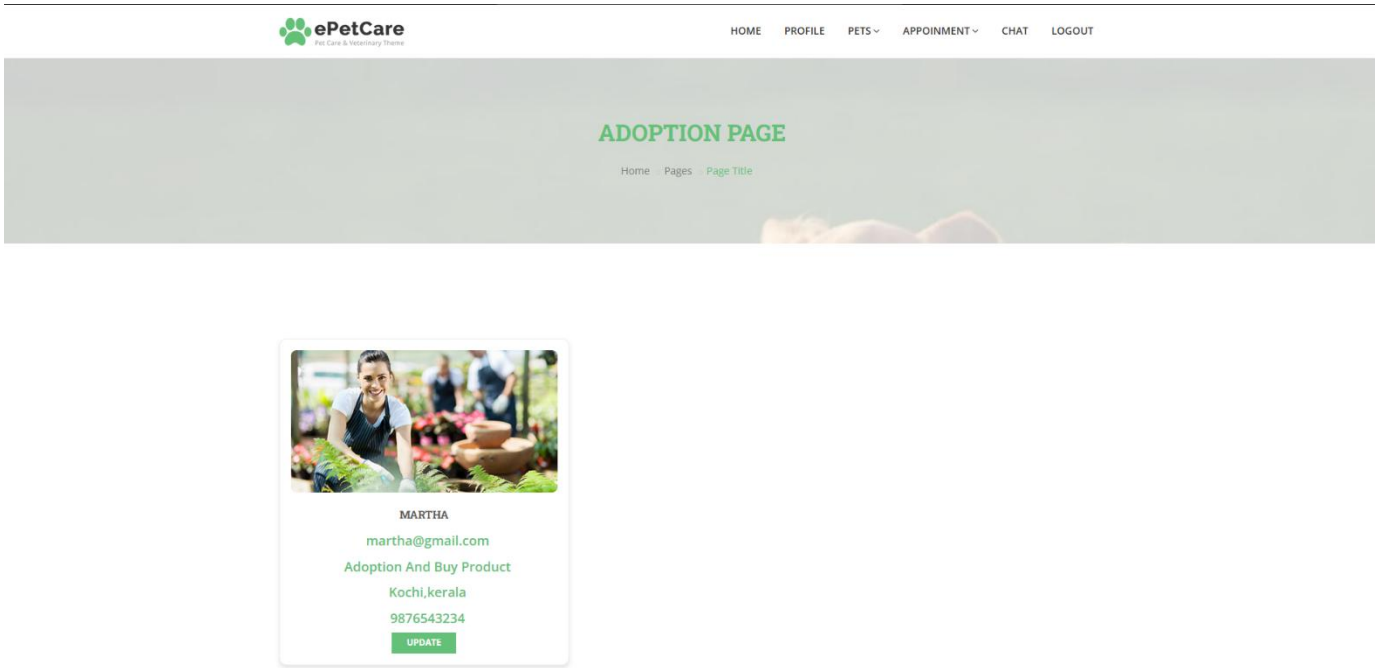


Stray dogs and cats

Chat Table:



Adoption Home Page:



8.3 APPENDIX C. SOURCE CODE

1.LOGIN.HTML

```
 {% extends "common.html" %}
{% load static %}
{% block content %}

{% for m in messages %}
   <script>
      alert('{{m}}');
   </script>
{% endfor %}

   <div class="main-content">

      <!-- Section: inner-header -->
      {% comment %} <section class="inner-header divider parallax layer-overlay overlay-white-8" data-
bg-img="{% static 'images/bg/bg6.jpg' %}">
      <div class="container pt-60 pb-60">
         <!-- Section Content -->
         <div class="section-content">
         <div class="row">
            <div class="col-md-12 text-center">
            <h2 class="title">LOGin</h2>
            <ol class="breadcrumb text-center text-black mt-10">
               <li><a href="">Home</a></li>
               <li><a href="">Pages</a></li>
               <li class="active text-theme-colored">Page Title</li>
            </ol>
            </div>
         </div>
         </div>
      </div>
      </section> {% endcomment %}

      <section>
      <div class="container">
         <div class="row">
         <div class="col-md-6 col-md-push-3">
            <div class="border-1px p-25">
            <h4 class="text-theme-colored text-uppercase m-0">Login</h4>
            <div class="line-bottom mb-30"></div>

            <form id="appointment_form" name="appointment_form" class="mt-30" method="post"
action="">
```

```
                {% csrf_token %}
                <div class="row">
                {% comment %} <div class="col-sm-12">
                   <div class="form-group mb-10">
                   <input name="form_name" class="form-control" type="text" required="" 
placeholder="Enter Name" aria-required="true">
                   </div>
                </div> {% endcomment %}
                <div class="col-sm-12">
                   <div class="form-group mb-10">
                   <input name="email" class="form-control required email" type="text" placeholder="Enter 
Username or Email" aria-required="true" required>
                   </div>
                </div>
                <div class="col-sm-12">
                   <div class="form-group mb-10">
                   <input name="password" class="form-control required" type="password" 
placeholder="Enter password" aria-required="true" required>
                   </div>
                </div>
                {% comment %} <div class="col-sm-12">
                   <div class="form-group mb-10">
                   <input name="form_appontment_date" class="form-control required date-picker" 
type="text" placeholder="Appoinment Date" aria-required="true">
                   </div>
                </div> {% endcomment %}
                {% comment %} <div class="col-sm-12">
                   <div class="form-group mb-10">
                   <input name="form_appontment_time" class="form-control required time-picker" 
type="text" placeholder="Appoinment Time" aria-required="true">
                   </div>
                </div> {% endcomment %}
                </div>

                <div class="form-group mb-0 mt-20">
                <input name="form_botcheck" class="form-control" type="hidden" value="">
                <button type="submit" class="btn btn-dark btn-theme-colored" data-loading-text="Please 
wait...">Submit</button>
                </div>
             </form>
             </div>
          </div>
          </div>
       </div>
       </section>
    </div>
```

```
 {% endblock %}
```

2.USERREG.HTML

```
{% extends "common.html" %}
{% load static %}
{% block content %}

{% for m in messages %}
<script>
   alert('{{m}}');
</script>
{% endfor %}

<div class="main-content">
   <div class="container">
      <div class="row">
         <div class="col-md-6 col-md-push-3">

            <div class="border-1px p-25">
               <h4 class="text-theme-colored text-uppercase m-0">Register</h4>
               <div class="line-bottom mb-30"></div>

               <form id="form" method="post" enctype="multipart/form-data">
                  {% csrf_token %}

                  <div class="row">

                     <!-- Name -->
                     <div class="col-sm-12">
                        <div class="form-group mb-10">
                           <input name="name"
                              class="form-control"
                              type="text"
                              placeholder="Enter Name"
                              required
                              minlength="3"
                              maxlength="40"
                              pattern="^[A-Za-z ]+$"
                              title="Name should contain only letters">
                        </div>
                     </div>

                     <!-- Email -->
                     <div class="col-sm-12">
                        <div class="form-group mb-10">
                           <input name="email"
```

```
                    class="form-control"
                    type="email"
                    placeholder="Enter Email"
                    required
                    title="Enter a valid email address">
                </div>
            </div>

            <!-- Password -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="password"
                        class="form-control"
                        type="password"
                        placeholder="Enter Password"
                        required
                        minlength="6"
                        pattern="^(?=.[A-Za-z])(?=.\d)[A-Za-z\d@$!%*#?&]{6,}$"
                        title="Password must contain at least 6 characters including one letter and one
number">
                </div>
            </div>

            <!-- Address -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="address"
                        class="form-control"
                        type="text"
                        placeholder="Enter Address"
                        required
                        minlength="5"
                        maxlength="150">
                </div>
            </div>

            <!-- Phone -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="phone"
                        class="form-control"
                        type="tel"
                        placeholder="Enter Phone"
                        required
                        pattern="[6-9][0-9]{9}"
                        maxlength="10"
```

```
                    title="Enter a valid 10 digit phone number">
                </div>
            </div>

            <!-- Category -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="category"
                        class="form-control"
                        type="text"
                        placeholder="Enter or Select Category"
                        list="category-list"
                        required>
                    <datalist id="category-list">
                        <option value="Adoption And Buy Product">
                        <option value="Rehoming">
                    </datalist>
                </div>
            </div>

            <!-- Image -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="image"
                        class="form-control"
                        type="file"
                        required
                        accept="image/png, image/jpeg, image/jpg"
                        title="Upload JPG or PNG image only">
                </div>
            </div>

        </div>

        <div class="form-group mt-20">
            <button type="submit"
                class="btn btn-dark btn-theme-colored">
                Submit
            </button>
        </div>

    </form>

  </div>

</div>
```

```
        </div>
    </div>
</div>

{% endblock %}
```

3.VETREG.HTML

```
{% extends "common.html" %}
{% load static %}
{% block content %}

{% for m in messages %}
<script>
    alert('{{m}}');
</script>
{% endfor %}

<div class="main-content">

    <section>
        <div class="container">
            <div class="row">
                <div class="col-md-6 col-md-push-3">

                    <div class="border-1px p-25">
                        <h4 class="text-theme-colored text-uppercase m-0">Vet Registration</h4>
                        <div class="line-bottom mb-30"></div>

                        <form id="form" method="post" enctype="multipart/form-data">
                            {% csrf_token %}

                            <div class="row">

                                <!-- Name -->
                                <div class="col-sm-12">
                                    <div class="form-group mb-10">
                                        <input name="name"
                                            class="form-control"
                                            type="text"
                                            placeholder="Enter Name"
                                            required
                                            minlength="3"
                                            maxlength="40"
                                            pattern="^[A-Za-z ]+$"
```

```
                    title="Name should contain only letters (min 3 characters)">
                </div>
            </div>

            <!-- Email -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="email"
                        class="form-control"
                        type="email"
                        placeholder="Enter Email"
                        required
                        title="Enter a valid email address">
                </div>
            </div>

            <!-- Password -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="password"
                        class="form-control"
                        type="password"
                        placeholder="Enter Password"
                        required
                        minlength="6"
                        pattern="^(?=.[A-Za-z])(?=.\d)[A-Za-z\d@$!%*#?&]{6,}$"
                        title="Password must contain at least 6 characters including one letter and one
number">
                </div>
            </div>

            <!-- Qualification -->
            <div class="col-sm-12">
                <div class="form-group mb-10">
                    <input name="qualification"
                        class="form-control"
                        type="text"
                        placeholder="Enter Qualification"
                        required
                        minlength="2"
                        maxlength="100">
                </div>
            </div>

            <!-- Experience -->
            <div class="col-sm-12">
```

```
            <div class="form-group mb-10">
                <input name="experience"
                    class="form-control"
                    type="number"
                    placeholder="Years of Experience"
                    required
                    min="0"
                    max="50"
                    title="Enter valid years of experience">
            </div>
        </div>

        <!-- Phone -->
        <div class="col-sm-12">
            <div class="form-group mb-10">
                <input name="phone"
                    class="form-control"
                    type="tel"
                    placeholder="Enter Phone"
                    required
                    pattern="[6-9][0-9]{9}"
                    maxlength="10"
                    title="Enter a valid 10 digit Indian phone number">
            </div>
        </div>

        <!-- Image -->
        <div class="col-sm-12">
            <div class="form-group mb-10">
                <input name="image"
                    class="form-control"
                    type="file"
                    required
                    accept="image/png, image/jpeg, image/jpg"
                    title="Upload JPG or PNG image only">
            </div>
        </div>

    </div>

    <div class="form-group mb-0 mt-20">
        <button type="submit"
            class="btn btn-dark btn-theme-colored">
            Submit
        </button>
    </div>
```

```
                    </form>

                  </div>

              </div>
            </div>
        </div>
    </section>

</div>

{% endblock %}
```

4.MODELS.PY

```
from django.db import models
from django.contrib.auth.models import AbstractUser

# Create your models here.
class Login(AbstractUser):
    usertype=models.CharField(max_length=50,null=True)
    view_password=models.CharField(max_length=50,null=True)


class User(models.Model):
    user=models.ForeignKey(Login,on_delete=models.CASCADE)
    name=models.CharField(max_length=50,null=True)
    password=models.CharField(max_length=50,null=True)
    email=models.EmailField()
    address=models.CharField(max_length=30,null=True)
    phone=models.CharField(max_length=50,null=True)
    category=models.CharField(max_length=50,null=True)
    image = models.FileField(upload_to='file', null=True)


class Vet(models.Model):
    vet=models.ForeignKey(Login,on_delete=models.CASCADE)
    name=models.CharField(max_length=50,null=True)
    password=models.CharField(max_length=50,null=True)
    email=models.EmailField()
    qualification=models.CharField(max_length=50,null=True)
    experience=models.CharField(max_length=50,null=True)
    phone=models.CharField(max_length=50,null=True)
    image = models.FileField(upload_to='file', null=True)
```

```
    status=models.CharField(max_length=20,default='Pending')



class Carecenter(models.Model):
    care=models.ForeignKey(Login,on_delete=models.CASCADE,null=True)
    name=models.CharField(max_length=50,null=True)
    password=models.CharField(max_length=50,null=True)
    email=models.EmailField()
    address=models.CharField(max_length=30,null=True)
    phone=models.CharField(max_length=50,null=True)
    image = models.FileField(upload_to='file', null=True)
    # year=models.CharField(max_length=200,null=True)
    careLicense=models.CharField(max_length=200,null=True)


class Pet(models.Model):
    care=models.ForeignKey(Carecenter,on_delete=models.CASCADE,null=True)
    user=models.ForeignKey(User,on_delete=models.CASCADE,null=True)
    name=models.CharField(max_length=50,null=True)
    species = models.CharField(max_length=50,null=True)
    breed = models.CharField(max_length=100, blank=True, null=True)
    age = models.CharField(max_length=50,null=True)
    gender = models.CharField(max_length=10,null=True)
    description=models.CharField(max_length=300,null=True)
    img=models.ImageField(upload_to="images",null=True)
    status = models.CharField(max_length=20,default='ForAdoption')
    Stray_status = models.CharField(max_length=20,default='Stray_dog&cat')



class Appointment(models.Model):
    STATUS_CHOICES = [
        ('pending', 'Pending'),
        ('scheduled', 'Scheduled'),
        ('completed', 'Completed'),
        ('rejected', 'Rejected'),
    ]

    user = models.ForeignKey(User, on_delete=models.CASCADE)
    vet = models.ForeignKey(Vet, on_delete=models.CASCADE)
    appointment_date = models.DateField()
    pet = models.ForeignKey(Pet, on_delete=models.CASCADE, null=True, blank=True)
    pet_species = models.CharField(max_length=50, null=True, blank=True)
```

```
    pet_breed = models.CharField(max_length=100, null=True, blank=True)
    time_slot = models.CharField(max_length=20, null=True, blank=True)  # Vet assigns this
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')

    def _str_(self):
        return f"Appointment with {self.vet.name} on {self.appointment_date} - {self.status}"



class Adoption(models.Model):
    user=models.ForeignKey(User,on_delete=models.CASCADE)
    pet=models.ForeignKey(Pet,on_delete=models.CASCADE)
    date = models.DateTimeField(auto_now_add=True)
    status = models.CharField(max_length=20,default='available')



class Chat(models.Model):
    sellerid = models.ForeignKey(User, on_delete=models.CASCADE)
    customerid = models.ForeignKey(Vet, on_delete=models.CASCADE)
    message = models.CharField(max_length=300)
    date = models.DateField(auto_now=True)
    time = models.CharField(max_length=100)
    utype = models.CharField(max_length=100)
```

9.BIBLIOGRAPHY

The development of the Stray Dogs and Cats Management System was completed with the help of various learning resources, textbooks, and reference materials. The Django framework documentation was used extensively to understand the concepts of models, views, templates, authentication, and database handling. Python programming language materials were referred to for learning core programming concepts, object-oriented programming, and backend logic implementation.

Web development technologies such as HTML, CSS, and JavaScript were studied from standard tutorials and academic resources to design and improve the user interface of the system. Additional guidance was taken from online programming communities and discussion platforms to resolve errors and understand better coding practices during development.

Reference implementations of similar Django-based projects helped in understanding project structure, module design, and real-time application flow. Academic lecture notes and classroom materials also played an important role in system analysis, design methodologies, and documentation preparation.

Overall, these resources collectively contributed to the successful completion of the project by improving technical knowledge, problem-solving skills, and system development understanding.